

lecture_24

October 2, 2026

1 Lecture 24: Simulated Annealing - Motivation & Pseudocode

{note} Lecture 23 argued that combinatorial transportation problems have no gradient to follow and too many candidate solutions to search exhaustively. The simplest fix - **local search** - starts from one solution and repeatedly moves to a better neighbouring solution, stopping when no neighbour improves further. This lecture shows why that simplest fix fails (it gets trapped at the first local optimum it meets) and introduces **Simulated Annealing (SA)**, which fixes it by occasionally accepting a **worse** move, with a controlled, decreasing probability inspired by how metals cool.

Learning Objectives

By the end of this notebook, you will be able to: 1. Explain why plain hill-climbing local search gets trapped at local optima, and how Simulated Annealing's probabilistic acceptance rule avoids this. 2. Read and interpret Simulated Annealing's pseudocode, including the role of the cooling schedule. 3. Hand-trace Simulated Annealing through several iterations given a solution landscape, a neighbourhood, and a sequence of random draws.

Prerequisites: Metaheuristics (Lecture 23).

Estimated time: 50 minutes

1.1 Why Local Search?

Local search is the most direct way to turn “explore and exploit a solution landscape” into an algorithm: start from an initial solution s_o , repeatedly look at solutions in its **neighbourhood** $N(s)$ — solutions reachable by one small change, such as swapping the order of two stops on a route — and move to a neighbour if it improves the objective f . The simplest version of this rule, **hill climbing**, moves to a neighbour *only if it's better*, and stops once no neighbour improves.

Hill climbing is cheap and easy to reason about, but it has a fatal flaw: once every neighbour of the current solution is worse, hill climbing stops — even if a much better solution exists a few moves further away, behind a temporary “hill” of worse solutions. The hand-traced example later in this lecture makes this concrete: a landscape with both a *local* optimum (better than all its immediate neighbours) and a *global* optimum (better than everything), separated by exactly one worse solution. Hill climbing, started at the local optimum, never leaves it.

{note} **Simulated Annealing** fixes this with one change to the acceptance rule: instead of accepting *only* improving moves, SA also accepts a worsening move with a probability that depends on how much worse it is and on a control parameter called **temperature** (T). Early in the search, when T is high, SA accepts worsening moves fairly often – enough to climb out of local optima. As T decreases according to a **cooling schedule**, SA accepts fewer and fewer worsening moves, eventually behaving like hill climbing and settling into a (hopefully much better) solution.

The name and the acceptance rule both come from **annealing** in metallurgy: a material is heated until its atoms move freely, then cooled slowly, so the atoms have time to settle into a low-energy, stable crystalline structure rather than getting frozen into a defect-ridden one. Cooling too fast (“quenching”) traps the material in a poor structure — the metallurgical analogue of hill climbing getting trapped at a local optimum.

1.2 Notation

Symbol	Meaning
s	Current solution
s^*	Best solution found so far
s'	A candidate solution generated from the neighbourhood of s
$N(s)$	Neighbourhood of s — the set of solutions reachable by one small change to s
$f(s)$	Objective function value (to be minimized) at solution s
T_k	Temperature at iteration k
T_o	Initial temperature
r	Cooling rate ($0 < r < 1$ for exponential cooling, $T_k = T_o r^k$)
$\Lambda \sim U(0, 1)$	A random number drawn uniformly from $[0, 1]$, used to decide whether to accept a worsening move

Besides exponential cooling ($T_k = T_o r^k$), common cooling schedules include **linear** ($T_k = T_o - \theta k$), **logarithmic** ($T_k = T_o / \log(k + 1)$), and **adaptive** schedules that speed up or slow down cooling based on how the search is progressing. A slow cooling schedule spends longer at higher temperatures — more exploration, less exploitation — while a fast cooling schedule does the opposite. Lecture 26 studies this tradeoff directly.

1.3 Pseudocode

1. **Procedure** $\text{SA}(s_o, N, f, T_o, r)$
2. $s \leftarrow s_o$ // initialise the current solution to the initial solution

```

3.  $s^* \leftarrow s$  // initialise the best solution to the current solution
4.  $k \leftarrow 0$  // initialise the iteration counter
5.  $T \leftarrow T_o$  // initialise the temperature
6. while !converged do // repeat until converged
7.    $s' \xleftarrow{R} N(s)$  // generate a random neighbour of the current solution
8.    $\Lambda \sim U(0, 1)$ ; draw  $\lambda$  // draw a random number, used only if  $s'$  is worse
9.   if  $f(s') < f(s)$  then // the neighbour is better
10.     $s \leftarrow s'$  // always accept an improving move
11.   else if  $\lambda < \exp(-(f(s') - f(s))/T)$  then // accept a worsening move with probability
        $\exp(-\Delta/T)$ 
12.     $s \leftarrow s'$  // accept anyway
13.   end if
14.   if  $f(s) < f(s^*)$  then // the current solution improves on the best found so far
15.     $s^* \leftarrow s$ 
16.   end if
17.    $k \leftarrow k + 1$ ;  $T \leftarrow T_o r^k$  // advance the iteration counter and cool the temperature
18. end while
19. return  $s^*$ 

```

{tip} Line 11's acceptance probability, $\exp(-\Delta/T)$ with $\Delta = f(s') - f(s) > 0$, is the **Boltzmann acceptance criterion**: it is always between 0 and 1, decreases as the move gets worse (larger Δ), and decreases as the temperature cools (smaller T). At $T \rightarrow \infty$ almost any move is accepted (pure random search); at $T \rightarrow 0$, only improving moves are accepted (pure hill climbing). SA's behaviour smoothly interpolates between these two extremes as the cooling schedule runs.

1.4 Hand-Traced Example

Consider a tiny landscape of five integer solutions, $s \in \{1, 2, 3, 4, 5\}$, with neighbourhood $N(s) = \{s - 1, s + 1\}$ (restricted to $\{1, \dots, 5\}$) and objective values:

s	1	2	3	4	5
$f(s)$	8	3	5	1	6

Solution $s = 2$ is a **local** optimum ($f(1) = 8$ and $f(3) = 5$ are both worse), but $s = 4$ is the **global** optimum. A hill-climbing search started at $s = 2$ would terminate immediately — every neighbour is worse. Trace Simulated Annealing instead, started at $s_o = 2$, with $T_o = 2$, cooling rate $r = 0.7$ ($T_k = 2 \times 0.7^k$), and the following pre-drawn random choices (both which neighbour is proposed, and the acceptance draw λ when needed):

k	s (before)	T_k	s' proposed	$f(s') - f(s)$	Rule	λ	Accept?	s (after)	s^*
0	2	2.00	3	$5 - 3 = +2$	$\lambda < \exp(-2/2) = 0.368?$	0.30	Yes (0.30 < 0.368)	3	2
1	3	1.40	4	$1 - 5 = -4$	improving— always accept	—	Yes	4	4
2	4	0.98	5	$6 - 1 = +5$	$\lambda < \exp(-5/0.98) = 0.0061?$	0.55	No (0.55 > 0.0061)	4	4

Two things to notice: - At $k = 0$, SA accepted a **worsening** move ($s = 2 \rightarrow s' = 3$, f went from 3 to 5) precisely because T was still high — this is the move that let it escape the local optimum at $s = 2$. Hill climbing would have stopped here forever. - By $k = 2$, the temperature has cooled enough that a similarly-sized worsening move ($\Delta = +5$) is rejected — SA is now behaving much more like hill climbing, exploiting rather than exploring. The algorithm found the global optimum $s^* = 4$ ($f = 1$) in three iterations, exactly because it was willing to *temporarily* accept a worse solution.

1.5 In-Class Exercise

1.5.1 Exercise 1 — Tracing From the Other Side

Using the same landscape and neighbourhood, trace Simulated Annealing started at $s_o = 5$, with $T_o = 3$, $r = 0.6$ ($T_k = 3 \times 0.6^k$), and pre-drawn choices: at $k = 0$ the proposal is $s' = 4$; at $k = 1$ the proposal is $s' = 3$, with acceptance draw $\lambda_1 = 0.20$.

k	s (before)	T_k	s' proposed	$f(s') - f(s)$	Rule	λ	Accept?	s (after)	s^*
0	5	3.00	4	$1 - 6 = -5$	improving— always accept	—	Yes	4	4
1	4	1.80	3	$5 - 1 = +4$	$\lambda < \exp(-4/1.8) = 0.108?$	0.20	No (0.20 > 0.108)	4	4

Starting at $s_o = 5$, the very first move is already improving (f drops from 6 to 1), so the search reaches the global optimum $s^* = 4$ immediately at $k = 0$ without ever needing to accept a worsening move — unlike the lecture's main example, which needed one. This is worth noting explicitly: **whether SA needs to accept a worsening move at all depends on where the search starts**, not just on its parameters. A cooling schedule generous enough to escape local optima when needed costs nothing when it isn't.

1.6 Take-Away Exercises

1.6.1 Exercise 1 — A Deeper Trap

Extend the landscape to $s \in \{1, \dots, 7\}$ with $f = [8, 3, 5, 5, 5, 1, 6]$ (so the “hill” separating the local optimum at $s = 2$ from the global optimum at $s = 6$ is now three steps wide instead of one). Starting at $s_o = 2$ with $T_o = 2$, $r = 0.8$, decide whether the same style of random draws that worked in this lecture’s example (occasional accepted worsening moves early on) would still be enough to cross a wider hill, or whether T_o and r need to change. Explain your reasoning in terms of the Boltzmann acceptance probability.

1.6.2 Exercise 2 — Cooling Rate Intuition

Without running any code, sketch (by hand or in words) how the acceptance probability $\exp(-\Delta/T)$ for a fixed $\Delta = 3$ changes across $k = 0, 1, 2, \dots, 10$ under two cooling rates, $r = 0.99$ (slow) and $r = 0.80$ (fast), both with $T_o = 5$. At what iteration does each schedule’s acceptance probability first drop below 0.05? What does this predict about each schedule’s ability to escape a local optimum that requires crossing a $\Delta = 3$ hill late in the search?

1.7 Circling Back

- **Lecture 23 (Metaheuristics):** Simulated Annealing is this module’s example of the *local search* paradigm — a single current solution, refined move by move, with the Boltzmann rule supplying its exploration mechanism.
- **Lecture 12 (NLP Principles):** Lecture 12 showed that convexity is what guarantees a gradient-based method never gets trapped in anything but the global optimum. The hand-traced example here is the discrete, non-convex analogue of exactly the failure convexity ruled out — a landscape with a genuine local trap.

1.8 Moving Forward

- **Lecture 25 (Simulated Annealing: Algorithm):** implements this pseudocode in Python and applies it to the Ackley function — a first hands-on encounter bigger than a five-solution toy example, before Lecture 26 tests it on a genuine combinatorial routing problem.
-

1.9 Further Reading

- Kirkpatrick, S., Gelatt, C.D., and Vecchi, M.P. (1983). “Optimization by Simulated Annealing.” *Science*, 220(4598), 671-680 — the original paper introducing SA.
- Černý, V. (1985). “Thermodynamical Approach to the Traveling Salesman Problem: An Efficient Simulation Algorithm.” *Journal of Optimization Theory and Applications*, 45(1), 41-51 — independently derived SA applied to the TSP.
- Aarts, E. and Korst, J. (1989). *Simulated Annealing and Boltzmann Machines*. Wiley.
- Van Laarhoven, P.J.M. and Aarts, E.H.L. (1987). *Simulated Annealing: Theory and Applications*. Springer — cooling schedule design in depth.